Upstream warnings in sae-lens

Two warnings we cannot fix in our own code, and how we could help fix them upstream. Prepared 2026-10-11 for the Feature Monitor project.

Summary

	Warning 1: deprecated import	Warning 2: model_from_pretrained_kwargs
Raised by	sae_lens/load_model.py, line 7 (on `import sae_lens`)	sae_lens/saes/sae.py, line 256 (every time this SAE is loaded)
Effect on us	None today. It becomes an ImportError on TransformerLens 4.x, which is why sae-lens pins transformer-lens below 4.0.	None. We measured that our loading already matches what the SAE expects.
Already fixed upstream?	Being fixed: issue #738 and pull request #739 are open, not merged.	No. Unchanged on the main branch; no existing issue found.
Best contribution	Test PR #739 with our setup and post the results as a review comment. Do not open a duplicate.	Open an issue describing the five problems below; offer a small pull request.

Environment checked

Package	Installed	Latest on PyPI (2026-10-11)	Licence	Repository
sae-lens	6.54.0	6.54.0	MIT	github.com/decoderesearch/SAELens
transformer-lens	3.9.0	4.2.0	MIT	github.com/TransformerLensOrg/TransformerLens

Python 3.12.14, torch 2.14.1, CPU. Both libraries are open source under the MIT licence, so outside contributions are possible. Note that sae-lens 6.54.0 declares `transformer-lens>=2.16.1,<4.0.0`. That is why our project cannot move to TransformerLens 4.2.0 yet, even though it has been released.

Warning 1: deprecated transformer_lens.utils import

What we see

```
sae_lens/load_model.py:7: DeprecationWarning: The 'utils' module has been deprecated.
Please use 'transformer_lens.utilities' instead. Importing from utils.py will be
removed in TransformerLens 4.0.
```

Where it comes from

A **deprecation warning** is a library's way of saying "this still works, but it will be removed in a future version". TransformerLens moved its helper functions from `transformer_lens.utils` to `transformer_lens.utilities`, and the old module now warns whenever it is imported. sae-lens still imports three helpers from the old place:

```
# sae_lens/load_model.py, lines 7-11 (sae-lens 6.54.0)
from transformer_lens.utils import (
    USE_DEFAULT_VALUE,
    get_tokens_with_bos_removed,
    lm_cross_entropy_loss,
)
```

We confirmed that all three names exist in `transformer_lens.utilities` in 3.9.0. The file is reached as soon as anyone writes `import sae_lens`, because the package's `__init__.py` imports the training runners (`cache_activations_runner`, `llm_sae_training_runner`, `multi_sae_training_runner`), and they import `load_model`. We only load a pre-trained SAE, but we still get the warning.

Why we cannot fix it ourselves

The import is inside the installed library, not in our code. Editing files in `.venv` would be undone by the next `uv sync`, and no other user would benefit.

Upstream status

- **Issue #738** (open, 2026-09-22): reports that `import sae_lens` fails with TransformerLens 4.0, which removed `HookedTransformer` and `transformer_lens.utils`. Its description says sae-lens had no upper bound on transformer-lens. Our installed 6.54.0 does have the `<4.0.0` bound, so the pin appears to have been added since.
- **Pull request #739** (open, marked ready for review 2026-09-30): adds `sae_lens/transformer_lens_compat.py`, which imports the helpers from `transformer_lens.utilities` and falls back to `transformer_lens.utils` on versions older than 3.0. This removes Warning 1. It also adds `model_class_name="TransformerBridge"` and keeps the `<4.0.0` pin for now.
- GitHub reported the pull request as not merged, with merge state "dirty". That usually means it conflicts with the current main branch.

How we could contribute

- **Do not open a duplicate** issue or pull request; the fix already exists.
- **Test PR #739 against our real use**: GPT-2 Small loaded with `TransformerBridge`, the `gpt2-small-res-jb` SAE at `blocks.8.hook_resid_pre`. Run our test suite, check that the warning is gone, and confirm the features are unchanged (our regression test already compares activations). Then post the results as a review comment. Maintainers value independent confirmation from real users.
- If the author agrees, offer help resolving the merge conflicts.

How to reproduce

```
uv run python -W default -c "import sae_lens"
# prints the DeprecationWarning from sae_lens/load_model.py:7
```

Warning 2: "This SAE has non-empty model_from_pretrained_kwargs"

What we see

```
sae_lens/saes/sae.py:256: UserWarning:
This SAE has non-empty model_from_pretrained_kwargs.
For optimal performance, load the model like so:
model = HookedSAETransformer.from_pretrained_no_processing(..., **cfg.model_from_pretrained_kwargs)
```

Where it comes from

Each pre-trained SAE stores the settings that were used to load the model it was trained on. For `gpt2-small-res-jb` this is `{'center_writing_weights': True}`, a TransformerLens option that subtracts the mean from the vectors each layer writes into the residual stream. If the model is loaded differently, the activations the SAE sees could differ from those it was trained on, and its features could be less accurate. The warning is meant to prevent that. It is raised in `SAE.__init__` whenever that dictionary is non-empty:

```
# sae_lens/saes/sae.py, lines 255-262 (6.54.0; identical on main, 2026-10-11)
if cfg.metadata and cfg.metadata.model_from_pretrained_kwargs:
    warnings.warn(
        "\nThis SAE has non-empty model_from_pretrained_kwargs. "
        "\nFor optimal performance, load the model like so:\n"
        "model = HookedSAETransformer.from_pretrained_no_processing("
        "..., **cfg.model_from_pretrained_kwargs)",
        category=UserWarning,
        stacklevel=1,
    )
```

Problems we found

#	Problem	Evidence
a	**The suggested code refers to an attribute that does not exist.** It says `cfg.model_from_pretrained_kwargs`, but the value lives at `cfg.metadata.model_from_pretrained_kwargs`.	`hasattr(sae.cfg, "model_from_pretrained_kwargs")` is `False`; the metadata path returns the dictionary.
b	**It recommends a deprecated API.** `HookedSAETransformer` subclasses `HookedTransformer`, which TransformerLens deprecates in 3.x and removed in 4.0. The message does not mention `TransformerBridge`, the replacement.	MRO check: `HookedSAETransformer` inherits `HookedTransformer`. Using it emits TransformerLens's own deprecation warning.
c	**It fires even when the model is loaded correctly.** The SAE is constructed independently of the model, so it cannot know how the model was loaded, and it warns every time.	For this SAE: reconstruction error (FVU) was 0.1845 with both the default loader and the recommended one (1 prompt). TransformerBridge in compatibility mode gave bit-identical layer-8 activations to `HookedTransformer` (40 prompts).
d	**It points at the wrong line.** `stacklevel=1` attributes the warning to `sae.py` itself, not to the user's `SAE.from_pretrained(...)` call, so it is hard to tell which load triggered it.	Our pytest output reports `sae_lens/saes/sae.py:256`, not our `monitor/model.py`.
e	**"For optimal performance" is unclear.** It means "so the activations match what the SAE was trained on" (accuracy), not speed.	Wording only.

Limits of our evidence. Problem (c) is shown for one SAE (`gpt2-small-res-jb`, layer 8) whose only setting matches the TransformerLens default. For SAEs with other settings, the warning's advice may be genuinely needed. The issue should say this, so that maintainers do not read it as "the warning is useless".

Why we cannot fix it ourselves

The warning is raised inside sae-lens while the SAE is being built. We could hide it with a warnings filter, but that only hides the symptom for us. A broad filter could also hide a real future warning, so we have deliberately not added one.

Upstream status

- Unchanged on the sae-lens main branch (checked 2026-10-11).
- No existing issue found. GitHub searches for `model_from_pretrained_kwargs` and `from_pretrained_no_processing warning` returned only unrelated or closed items.
- PR #739 changes one line of `sae.py`, an import, but not this warning.

How we could contribute

- **Step 1: open an issue first**, so the maintainers can choose the direction before anyone writes code. Describe problems (a) to (e) with the evidence above and our environment, and include the limits note.
- **Step 2: offer a small pull request**, following whatever the maintainers prefer. Candidate changes:
  - Correct the attribute path to `cfg.metadata.model_from_pretrained_kwargs`.
  - Recommend the loader that matches the installed TransformerLens version. `TransformerBridge` is the 3.x/4.x replacement; align this with the `model_class_name="TransformerBridge"` option from PR #739.
  - Raise `stacklevel` so that the warning points at the caller.
  - Reword "optimal performance" to say the setting keeps activations consistent with how the SAE was trained.
  - Optionally, include the actual settings in the message, for example `{'center_writing_weights': True}`, so users can check them.

How to reproduce

```
uv run python -W default -c "
from sae_lens import SAE
SAE.from_pretrained('gpt2-small-res-jb', 'blocks.8.hook_resid_pre')"
# prints the UserWarning from sae_lens/saes/sae.py:256
```

General steps for a first open-source contribution

- Read the repository's README and contributing guide (if it has one) for its development setup, test commands and code style.
- Search existing issues and pull requests first, as done above.
- Open an issue or comment before writing code, and keep any pull request small and focused on one problem.
- Include a minimal reproduction, exact versions, and tests for the change.
- Be patient and responsive to review. Maintainers are often volunteers.

Links

- sae-lens repository: https://github.com/decoderesearch/SAELens
- Issue #738: https://github.com/decoderesearch/SAELens/issues/738
- Pull request #739: https://github.com/decoderesearch/SAELens/pull/739
- TransformerLens repository: https://github.com/TransformerLensOrg/TransformerLens

All facts in this document were checked on 2026-10-11 against the installed packages, the GitHub API and PyPI. Upstream status can change; re-check the links before acting.